

Lineare Optimierung

Optimierung einer linearen Zielfunktion mit linearen Nebenbedingungen

Grafische Lösung

1. Isolinien der Zielfunktion zeichnen
2. Nebenbedingungen als Geraden einzeichnen
3. Zulässigen Bereich markieren
4. Zielfunktion parallel verschieben bis zur letzten Berührung mit dem zulässigen Bereich
5. Berührstelle ist die optimale Lösung

Standardformat für Lin. Optimierung

- nur Minimierungsprobleme
- nur Gleichungen ($\rightarrow$ kein Ungleichung)
- nur nicht-negative Variablen

Slack-Variablen

Werden zur Formung von Gleichungen genutzt

$$3x_1 + x_2 \geq 25 \rightarrow 3x_1 + x_2 + x_3 = 25$$

Problem: Negative Variablen

bei vorgegebenem Wertebereich $x_j \in [a; b]$ mit $a < 0$ einfach durch die Differenz $x_j = x_j - a$ ersetzen

Simplex-Algorithmus

- Beginnt an einem Eckpunkt des zulässigen Bereichs
- prüft, entlang welcher Kante die größte Verbesserung möglich ist
- bewegt sich von Ecke zu Ecke, bis keine Verbesserung mehr möglich ist

Integer-Problem

Manchmal sind nur ganze Zahlen als Lösung erlaubt

$\rightarrow$ Runden kann ungenau sein

$\rightarrow$ Cut einfügen, die von nicht-ganzzahligen Lösungen verletzt

Gradient Descent

1. Setze Startlösung $\vec{x}_{i=0}$
2. Gradienten berechnen
3. Wenn Grad $\vec{x} = [0, \dots, 0]^T \rightarrow$ beenden
4. Bewegung in Richtung des Gradienten mit $\vec{x}_{i=i+1} = \vec{x} - c \cdot \nabla f(\vec{x}_i)$
 $\rightarrow c$ ist ein Parameter des Algorithmus
5. Gehe zu 2

Transportproblem

Warentransport bei minimalen Kosten, von mehreren Anbietern zu mehreren Nachfragern, wobei das Gesamtangebot der Gesamtnachfrage entspricht

Matrix-notation für Problem

		Nachfrager				
		1	2	...	n	Angebot
Anbieter	1	x_{11}	x_{12}	...	x_{1n}	a_1
	2	x_{21}	x_{22}	...	x_{2n}	a_2
	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
	m	x_{m1}	x_{m2}	...	x_{mn}	a_m
Nachfrage		b_1	b_2	...	b_n	

Startlösung: Nord-West-Ecken-Regel (NWER)

1. Nordwestlichste leere Zelle füllen mit maximale möglicher Kapazität (Minimum von Angebot und Nachfrage). Verringere Angebot und Nachfrage entsprechend.
2. Falls Angebot erschöpft: fülle restliche Zeile mit Nullen (Das Gleiche bei Nachfrage und Spalte)
3. Schritt 1 und Schritt 2 bis Tabelle voll ist wdhn. Matrixminimummethode (MMM)

1. Suche leere Zelle mit geringsten Transportkosten, fülle mit maximal möglicher Kapazität. Verringere Angebot & Nachfrage.
2. Gleich mit NWER.
3. Schritt 1 und Schritt 2 bis Tabelle voll ist wdhn. Vogel-Methode (VM)

1. Opportunitätskosten (Ok) für jede Zeile/Spalte berechnen (zweitniedrigster - niedrigster Wert).
2. In Zeile/Spalte mit höchstem Ok die niedrigsten Kosten wählen und wie beim NWER oder MMM füllen.
3. Schritt 1 und Schritt 2 bis Tabelle voll ist wdhn.

Quadratische Optimierung

Optimierung einer quadratischen Zielfunktion mit linearen Nebenbedingungen

$$\text{minimiere } f(\vec{x}) = \frac{1}{2} \vec{x}^T Q \vec{x} + \vec{c}^T \vec{x}$$

$$\text{mit } g(\vec{x}) : A \vec{x} \leq b$$

$$h(\vec{x}) : A_{eq} \vec{x} = \vec{b}_{eq}$$

Q sollte symmetrisch sein $Q = Q^T$

Nicht-lineare Optimierung mit Nebenbedingungen
KKT-Bedingungen

$$\text{minimiere}_{\vec{x}} f(\vec{x})$$

$$\text{mit } h_k(\vec{x}) = 0$$

$$g_j(x) \leq 0$$

$$\text{Lagrange-Funktion } \Delta f(x) + \sum_{i=1}^k a_i \nabla_{\vec{x}} h_i(\vec{x}) + \sum_{j=1}^l b_j \nabla_{\vec{x}} g_j(\vec{x}) = 0$$

Bestimmung ob Minimum oder Maximum mithilfe der Definitheit der Hesse-Matrix

Sequentielle Quadratische Optimierung

Solver für nichtlineare Optimierung

Vorgehen

1. Setze Startlösung $x_{i=0}$

2. quadratische Funktion approximieren, Suchrichtung $\vec{d}$ finden

3. Schrittgröße c mittels Liniensuche bestimmen

4. Nächsten Lösungskandidat mittels $\vec{x}_{i+1} = \vec{x}_i + c \vec{d}$

5. Wenn nicht konvergiert zu Schritt 2

05 Gradientenfrei

Nelder-Mead Simplex

Simplex

1-D Simplex : Liniensegment

2-D Simplex : Dreieck

3-D Simplex : Tetraedron

Algorithmus

1. Erzeuge initialen Simplex $x_1, x_2, x_3, \dots, x_n$
2. Sortiere die Ecken nach Zielfunktionswert $f(\vec{x})$
3. Wenn Abbruchbedingung erfüllt : Stopp, sonst weiter
4. Berechne 'centerpoint' der besten Seite $\vec{c}$
5. Transformiere Simplex : a) reflect b) expand c) contract d) shrink

Initialisierung

Initiallösung $\vec{x}_0$

$$\vec{x}_j = \vec{x}_0 + h_j \vec{e}_j \quad \text{mit } j = 1, \dots, n$$

- Schrittweite h_j (oft sehr klein)
- j-th Einheitsvektor $\vec{e}_j$

Sortierung

Sortierung der Indizes $f(\vec{x}_0) \leq f(\vec{x}_1) \leq f(\vec{x}_2) \leq \dots \leq f(\vec{x}_n)$

Abbruch

Abbruch wenn Summe der Distanzen der Punkte unter Schwellenwert rutscht

centerpoint

centerpoint der Seite gegenüber $\vec{x}_n$ berechnen

$$\vec{c} = \frac{1}{n} \sum_{j=0}^{n-1} \vec{x}_j$$

5a reflect

- Erzeuge neuen Punkt mit Reflektion
- $\vec{x}_r = \vec{c} + \alpha \cdot (\vec{c} - \vec{x}_n)$
- wenn $f(\vec{x}_0) \leq f(\vec{x}_r) \leq f(\vec{x}_{n-1}) \rightarrow$ so semi gut
 - Ersetze schlechtesten, gehe zu Schritt 2
- wenn $f(\vec{x}_r) \leq f(\vec{x}_0) \rightarrow$ hat super funktioniert
 - weiter mit 5b expand
- wenn $f(\vec{x}_r) \geq f(\vec{x}_{n-1}) \rightarrow$ schlecht
 - weiter mit 5c contract

5b expand

- Erzeuge neuen Punkt mit Erweiterung
- $\vec{x}_e = \vec{c} + \gamma (\vec{x}_r - \vec{c})$
- wenn $f(\vec{x}_e) < f(\vec{x}_r)$
 - ersetze schlechtesten $\vec{x}_n = \vec{x}_e$
 - Gehe zu Schritt 2
- Sonst
 - Ersetze schlechtesten $\vec{x}_n = \vec{x}_r$
 - Gehe zu Schritt 2

5c contract

- Wenn $f(\vec{x}_{n-1}) \leq f(\vec{x}_r) \leq f(\vec{x}_n)$
 - kontrahiere mit $\vec{x}_r$
 - $\vec{x}_c = \vec{c} + \beta \cdot (\vec{x}_r - \vec{c})$
- Sonst, wenn $f(\vec{x}_r) \geq f(\vec{x}_n)$
 - kontrahiere mit $\vec{x}_n$
 - $\vec{x}_c = \vec{c} + \beta \cdot (\vec{x}_n - \vec{c})$
- Wenn $f(\vec{x}_c) < f(\vec{x}_n)$
 - Ersetze schlechtesten $\vec{x}_n = \vec{x}_c$
 - Gehe zu Schritt 2
- Sonst weiter mit 5d shrink

5d shrink

(Nur bei sehr komplexen Suchlandschaften)

- 3
- $\vec{x}_j = \vec{x}_0 + \delta \cdot (\vec{x}_j - \vec{x}_0)$ für alle $j > 0$
 - Gehe zu Schritt 2

Algorithmus: DIRECT (Dividing RECTangles)

1. Initialisierung
2. Bestimmung der Menge S von potentiell optimalen Rechtecken
3. Für jedes Rechteck $j \in S$: sample & divide
4. $t = t + 1$, wenn $t = T$ stopp, sonst gehe zu Schritt 3

Initialisierung

- Normalisierung des Suchraums zu unit-hypercube
- centerpoint $\vec{c}_0$ mit $f_{\min} = f(\vec{c}_0)$
- $m = 1$ Auswärtungszähler
- $t = 0$ Iterationszähler

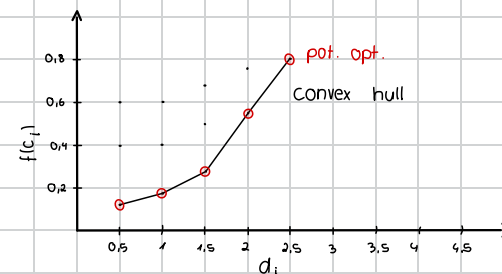
Bestimme potentiell optimale Rechtecke

- Ein centerpoint $\vec{c}_j$ ist pot. opt., wenn ein $\tilde{k} > 0$ existiert mit
 - $f(\vec{c}_j) - \tilde{k}d_j \leq f(\vec{c}_i) - \tilde{k}d_i$ für alle $i \in 1, \dots, m$
 - $f(\vec{c}_j) - \tilde{k}d_j \leq f_{\min} - \epsilon |f_{\min}|$
- d_j Distanz von centerpoint zur Ecke des Rechtecks
- ϵ kleine positive Zahl

Sample and Divide

- Bestimme Menge I von Dimensionen mit maximaler Seitenlänge 3δ
- Sample** (neue Punkte setzen) bei $\vec{c} \pm \delta \vec{e}_i$ für alle $i \in I$
- Divide** des Rechtecks in drei gleich große Drittel, beginnend mit $i \in I$ welches den besten Wert $f(\vec{c}_i)$ liefert

convex hull



06 Metaheuristiken

Metaheuristiken und Naturanaloge Algorithmen

- Metaheuristik: Gruppe von problemabhängigen, approximativen Solvern

Viele naturinspiriert:

- ↳ Simulated Annealing
- ↳ Particle Swarm Optimization
- ↳ Ant Colony Optimization
- ↳ Evolutionäre Algorithmen

Evolutionäre Algorithmen

DNA

- besteht aus Nukleotiden C G A T
 - Mutation: CGT **AG**TAC → CGT **C**G-TAC
 - Rekombination: **CG**-**ATAG**TAC + **AC**ATT**TGG** → **CG**TA**TTAG**
- bei binärer Optimierung deckgleich

Kontinuierliche Optimierung

Vektoren reeller Zahlen $\vec{x} = [0.1, 0.2, 1.1, -1.3]$

Mutation: $\vec{x} + \epsilon = [0.0, 0.1, 1.0, -1.4]$

Rekombination: $\vec{x}_1 + \vec{x}_2 / 2 = \vec{x}_3$

1+1 Evolution Strategy

Weg zum Umgang mit Auswirkungen von Algorithmus-Parametern

Motivation der 1/5-Rule:

- ↳ kriegt man zu oft bessere Lösungen, so ist die Schrittweite zu klein
- ↳ kriegt man fast gar keine guten Lösungen, so ist die Schrittweite zu groß

1/5 wegen 'öfter als 1 Erfolg bei 5 Versuchen' oder andersrum

Algorithmus

Parameter: Speicherlänge g , Schrittweitenmultiplikator $a > 1$, initiale Schrittweite $\vec{\sigma} = \vec{1}$

1. Startwert oder zufällige Startlösung $\vec{x}$, $y = f(\vec{x})$
2. Clone: kopieren der aktuellen Lösung $\vec{x}^{\text{next}} = \vec{x}$
3. Mutation: Addieren eines normalverteilten Zufallswertes zu jeder Variable: $\vec{x}_i^{\text{next}} = \vec{x}_i^{\text{next}} + \epsilon_i$
4. Auswertung: $y^{\text{next}} = f(\vec{x}^{\text{next}})$
5. Selektion: Wenn $y^{\text{next}} < y$: Erfolg (1) speichern in Liste g letzter Schritt G und $\vec{x} = \vec{x}^{\text{next}}$, $y = y^{\text{next}}$
Sonst: Fehlschlag (0) speichern. Wenn nötig ältesten Löschen
6. Wenn Erfolgsrate $s > 1/5$: $\vec{\sigma} = \vec{\sigma} \cdot a$
 $s < 1/5$: $\vec{\sigma} = \vec{\sigma} \cdot \frac{1}{a}$
7. Abbruchbedingung? Sonst Schritt 2

07 Mehrzieloptimierung

Pareto-optimal

Ein Punkt $\vec{p}^* \in \mathbb{R}$ ist optimal, wenn es keinen anderen Punkt $x \in \mathbb{R}$ gibt, der in Bezug auf alle Zielfunktionen gleich oder besser ist und in mindestens einer Funktion besser ist.

Pareto-Menge

Menge von Pareto-optimalen Vektoren

Pareto-Front

Darstellung der Pareto-Menge im Zielfunktionsraum

Wie sieht die Pareto-Menge aus

2 Ziele, 2 Variablen $\rightarrow$ Linie

3 Ziele, 2 Variablen $\rightarrow$ Fläche

Pareto-Dominanz

$\vec{x}'$ dominiert $\vec{x}''$ wenn:

$\hookrightarrow f_i(\vec{x}') \leq f_i(\vec{x}'')$ für alle $i \in \{1, \dots, m\}$

$\hookrightarrow f_i(\vec{x}') < f_i(\vec{x}'')$ für mindestens ein $i \in \{1, \dots, m\}$

Non-Dominated Sorting Rank (NDSA)

Gegeben: Menge von Punkten (im Zielfunktionsraum),
Start Rang $i = 1$

1. Alle nicht dominierten Punkte erhalten den Rang i

2. Die Punkte mit Rang werden aus der berücksichtigten Punktemenge entfernt

3. $i = i + 1$

Punkte 1 bis 3 solange wiederholen, bis alle Punkte einen Rang erhalten haben.

NSGA-II

Crowding Distance entscheidet bei Gleichstand

Crowding Distance

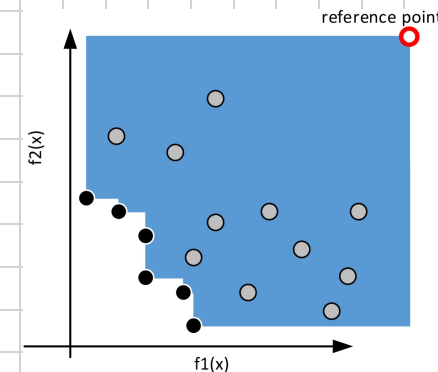
nur für Punkte auf derselben Front berechnen

basierend auf den umschließenden Nachbarn

Summe der Abstände zwischen den Nachbarn in jedem Ziel

Skalierung: Teilen der Abstände durch den größten beobachteten Abstand zweier Punkte

Hypervolumen der Pareto-Front



Hypervolumenbeitrag

